

Handling Aperiodic Overloads — Buttazzo

Milestone 3: Critical Evaluation and Transfer

Rionela Kovaci

Hamm-Lippstadt University of Applied Sciences

Real-Time Systems Seminar

rionela.kovaci@stud.hshl.de

I. STRENGTHS

- RED protects the most important tasks even when the system is overloaded. Plain EDF cannot do this [1].
- The guarantee test runs in $O(n)$ time. This is fast enough to use at every task arrival without slowing the system down [1].
- Rejected tasks are not deleted permanently. They are kept in a queue and re-accepted when CPU time becomes available. This reduces unnecessary task losses.
- Deadline tolerance makes the algorithm less strict. A task that finishes slightly late can still be accepted. This reduces the number of unnecessary rejections.
- The algorithm separates guarantee, scheduling, and rejection into independent steps. This makes it easy to replace one part without changing the others.

II. LIMITATIONS AND RISKS

- RED only works on a single processor. It cannot be used on multicore systems without major changes [1].
- The worst-case execution time must be known when the task arrives. In many real systems this is hard to measure accurately.
- RED only handles aperiodic tasks. Periodic tasks running at the same time are not considered. Most real embedded systems have both.
- Task values must be assigned by the designer. There is no automatic way to decide which tasks are more important. Wrong values lead to wrong rejections.
- The algorithm uses four queues at runtime. On small microcontrollers with limited memory this can be a problem.
- If the overload is very large, even RED cannot help. When all tasks are CRITICAL, nothing can be rejected and the system still fails.

III. SCALABILITY AND IMPLEMENTATION

- Runtime complexity is $O(n)$ per arrival — scales well for small to medium task sets.
- Memory grows with the number of active and rejected tasks — problematic on constrained hardware.
- Queue management adds overhead at every task completion, not only at arrival.
- No implementation on a real RTOS is shown in the paper — only simulation results are provided.

IV. WHERE THE APPROACH IS SUITABLE

- **Robotics:** a robot controller with sensor events as aperiodic tasks. Some tasks (emergency stop) are CRITICAL, others (display update) are HARD. RED protects the emergency stop even under heavy load.
- **Research systems with EDF scheduler:** RED works well on systems already using EDF where aperiodic overloads are expected but rare.

V. WHERE THE APPROACH IS UNSUITABLE

- **FreeRTOS / AUTOSAR systems:** these use fixed-priority scheduling, not EDF. RED cannot be deployed directly.
- **Multicore embedded platforms:** RED has no multicore support. Modern automotive and industrial systems often use dual or quad-core processors.
- **Systems with unknown WCET:** if execution times cannot be estimated (e.g., network-dependent tasks), RED cannot guarantee correctness.

VI. OPEN QUESTIONS AND POSSIBLE EXTENSIONS

- Can RED be extended to handle periodic and aperiodic tasks together in one framework?
- How can task values be assigned automatically based on system requirements, without manual configuration?
- Can RED be adapted for fixed-priority schedulers so it works on FreeRTOS or AUTOSAR?
- How does RED behave on multicore systems? Can the guarantee test be parallelized?
- Can machine learning be used to predict overload before it happens, so RED can act earlier?

REFERENCES

- [1] G. C. Buttazzo and J. A. Stankovic, "Adding Robustness in Dynamic Preemptive Scheduling," Kluwer Academic Publishers, 1995.
- [2] M. Spuri and G. C. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," *Real-Time Systems*, vol. 10, pp. 179–210, 1996.
- [3] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.